**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

## I. General Statement of the Problem

a. What is the application about? Who are the target users? What problem does the application solve?

Le Festin is a recipe and meal plan management system designed to help users organize recipes, manage ingredients, and plan meals efficiently. The target users of the application include students, busy professionals, families, home cooks, and anyone interested in tracking their meals, recipes, or pantry ingredients in a convenient and organized way.

The application solves common problems such as difficulty in organizing recipes, forgetting available pantry ingredients, wasting food due to unused ingredients, and struggling to plan meals ahead of time. By combining recipe management, ingredient tracking, and meal planning features into one system, Le Festin helps users save time, reduce food waste, and make meal preparation more efficient and convenient.

## II. Database Requirements

**A. E/R Diagram**: Design of the database schema.

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

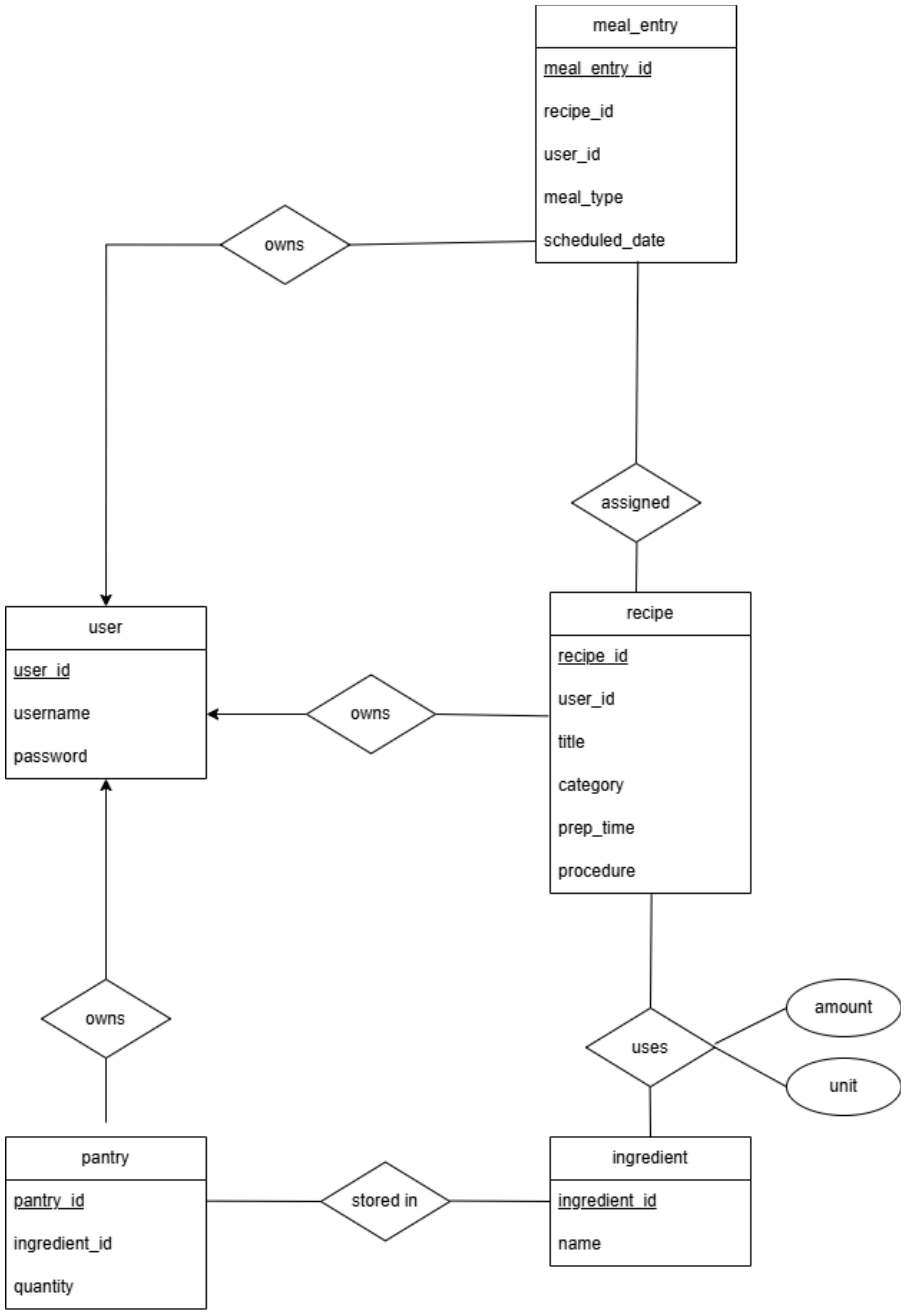


Figure 1. E/R Diagram of Le Festin

**B. Technical Description**: Explanation of the database requirements and how they relate to the E/R diagram.

**1. Structural Hierarchy and User Ownership**

The system employs a centralized ownership architecture where the User entity serves as the primary anchor for all data.

Requirement Implementation: The narrative specifies that each user maintains isolated collections. The User entity (containing user_id, username, and password) is connected via "owns" relationship diamonds to the Recipe, Pantry, and Meal_Entry entities. This ensures that while a single user can possess multiple records across the system, each record is strictly mapped to one unique identifier, preventing cross-user data leakage.

**2. Recipe Specification and Content**

The Recipe entity functions as a static template for culinary instructions.

Requirement Implementation: The system requires unique identification and descriptive metadata, including title, category, and preparation instructions. These are modeled as attributes of the Recipe entity: recipe_id, title, category, prep_time, and procedure.

**3. Ingredient Composition**

The relationship between Recipe and Ingredient represents the most complex logical intersection in the schema.

Requirement Implementation: Ingredients must be reusable across multiple recipes, and recipes must support multiple ingredients. Additionally, the system must record unique measurements (quantity and unit) for each instance. The amount and unit are placed as attributes on the relationship itself rather than the entities. This signifies that these values only exist when a specific ingredient is assigned to a specific recipe.

**4. Pantry Inventory and Stock Management**

The Pantry entity acts as a junction to track physical inventory without duplicating master ingredient data.

Requirement Implementation: The pantry must link a specific user to a specific ingredient while tracking current volume. The Pantry entity contains the quantity attribute and is linked to the User via an "owns" relationship and to the Ingredient via the "stored in" relationship. This design allows the system to differentiate between a Master Ingredient (the concept of salt) and a Pantry Item (the 500g of salt in a user's cabinet).

**5. Dynamic Meal Planning and Overrides**

The Meal_Entry entity provides a temporal layer to the database, allowing for scheduling and categorization.

Requirement Implementation: Users must organize recipes into daily, weekly, or monthly plans and have the ability to redefine the meal type for that specific instance. The Meal_Entry entity contains meal_type and scheduled_date.

C. **Relation Schema**: Conversion of the E/R diagram into relational schemas, including primary and foreign keys.

SQL

**user(<u>user_id</u>, username, password_hash)** PK: user_id
PK: user_id

**ingredient(<u>ingredient_id</u>, name)** PK: ingredient_id
PK: ingredient_id

**recipe(<u>recipe_id</u>, user_id, title, category, prep_time, procedure)**
PK: recipe_id
FK: user_id → user(user_id)

**recipe_ingredient(<u>recipe_id</u>, <u>ingredient_id</u>, quantity, unit)**
PK: recipe_id, ingredient_id
FK: recipe_id → recipe(recipe_id), ingredient_id → ingredient(ingredient_id)

**pantry(<u>ingredient_id</u>, <u>user_id</u>, quantity, unit)**
PK: ingredient_id, user_id
FK: ingredient_id → ingredient(ingredient_id), user_id → user(user_id)

**meal_entry(<u>user_id</u>, <u>meal_type</u>, <u>scheduled_date</u>, recipe_id)**
PK: user_id, meal_type, scheduled_date
FK: recipe_id → recipe(recipe_id), user_id → user(user_id)

## III. Implementation

A. **SQL Table Definition**

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

```sql
-- ============================================================
--  Le Festin — Full Schema Setup
--  Safe to re-run: drops and recreates all tables cleanly.
--  Run with:
--  mysql -u root -p < sql/la_festin_schema.sql
-- ============================================================

-- ── Database ──────────────────────────────────────────────────
CREATE DATABASE IF NOT EXISTS le_festin
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE le_festin;

-- ── Safety: drop in reverse dependency order ──────────────────
-- Child tables first, parent tables last.
-- Prevents FK constraint errors on re-run.
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS meal_entry;
DROP TABLE IF EXISTS pantry;
DROP TABLE IF EXISTS recipe_ingredient;
DROP TABLE IF EXISTS recipe;
DROP TABLE IF EXISTS ingredient;
DROP TABLE IF EXISTS user;
SET FOREIGN_KEY_CHECKS = 1;

-- ============================================================
--  TABLE 1: user
--  No dependencies — created first.
-- ============================================================
-- Enforcing unique usernames
CREATE TABLE user (
  user_id      INT          NOT NULL AUTO_INCREMENT,
  username     VARCHAR(50)  NOT NULL,
  password_hash VARCHAR(255) NOT NULL,

  CONSTRAINT pk_user     PRIMARY KEY (user_id),
  CONSTRAINT uq_username UNIQUE      (username)
);

-- ============================================================
--  TABLE 2: ingredient
--  No dependencies — created alongside user.
-- ============================================================
-- Enforcing unique ingredient names such that it does not get added to the DB if
-- there is an existing one
```

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

```
--
CREATE TABLE ingredient (
  ingredient_id INT         NOT NULL AUTO_INCREMENT,
  name          VARCHAR(100) NOT NULL,

  CONSTRAINT pk_ingredient PRIMARY KEY (ingredient_id),
  CONSTRAINT uq_ingredient UNIQUE      (name)
);

-- ============================================================
--  TABLE 3: recipe
--  Depends on: user
-- ============================================================
CREATE TABLE recipe (
  recipe_id  INT        NOT NULL AUTO_INCREMENT,
  user_id    INT        NOT NULL,
  title      VARCHAR(150) NOT NULL,
  category   ENUM(
               'Breakfast',
               'Lunch',
               'Dinner'
             )          NOT NULL,
  prep_time  INT        NOT NULL,
  `procedure`  TEXT        NOT NULL,

  CONSTRAINT pk_recipe      PRIMARY KEY (recipe_id),
  CONSTRAINT fk_recipe_user  FOREIGN KEY (user_id)
    REFERENCES user (user_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT chk_prep_time   CHECK (prep_time > 0)
);

-- ============================================================
--  TABLE 4: recipe_ingredient
--  Depends on: recipe, ingredient
-- ============================================================
CREATE TABLE recipe_ingredient (
  recipe_id    INT          NOT NULL,
  ingredient_id INT          NOT NULL,
  quantity     DECIMAL(10, 2) NOT NULL,
  unit         VARCHAR(50)    NOT NULL,

  CONSTRAINT pk_recipe_ingredient
    PRIMARY KEY (recipe_id, ingredient_id),
```

```
  CONSTRAINT fk_ri_recipe
    FOREIGN KEY (recipe_id)
    REFERENCES recipe (recipe_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_ri_ingredient
    FOREIGN KEY (ingredient_id)
    REFERENCES ingredient (ingredient_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT chk_ri_quantity CHECK (quantity > 0)
);

-- ============================================================
--  TABLE 5: pantry
--  Depends on: ingredient, user
-- ============================================================
CREATE TABLE pantry (
  ingredient_id INT          NOT NULL,
  user_id       INT          NOT NULL,
  quantity      DECIMAL(10, 2) NOT NULL,
  unit          VARCHAR(50)    NOT NULL,

  CONSTRAINT pk_pantry PRIMARY KEY (ingredient_id, user_id),
  CONSTRAINT fk_pantry_ingredient
    FOREIGN KEY (ingredient_id)
    REFERENCES ingredient (ingredient_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_pantry_user
    FOREIGN KEY (user_id)
    REFERENCES user (user_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT chk_pantry_qty CHECK (quantity >= 0)
);

-- ============================================================
--  TABLE 6: meal_entry
--  Depends on: recipe, user
-- ============================================================
CREATE TABLE meal_entry (
  recipe_id     INT  NOT NULL,
  user_id       INT  NOT NULL,
  meal_type     ENUM(
                'Breakfast',
```

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

```
            'Lunch',
            'Dinner'
          )    NOT NULL,
  scheduled_date DATE NOT NULL,

  CONSTRAINT pk_meal_entry
    PRIMARY KEY (user_id, scheduled_date, meal_type),
  CONSTRAINT fk_me_recipe
    FOREIGN KEY (recipe_id)
    REFERENCES recipe (recipe_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_me_user
    FOREIGN KEY (user_id)
    REFERENCES user (user_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);

-- ============================================================
--  INDEXES
-- ============================================================
CREATE INDEX idx_recipe_user  ON recipe        (user_id);
CREATE INDEX idx_ri_recipe    ON recipe_ingredient(recipe_id);
CREATE INDEX idx_pantry_user  ON pantry        (user_id);
CREATE INDEX idx_me_user_date ON meal_entry    (user_id, scheduled_date);

-- ============================================================
--  DONE
-- ============================================================
SELECT 'le_festin schema created successfully.' AS status;


-- ============================================================
--  Le Festin — Unit Normalization Migration
--
--  Purpose:
--  Align existing pantry and recipe_ingredient rows with the
--  canonical storage units now used by ConversionService:
--    - MASS   -> gram
--    - VOLUME -> milliliter
--    - COUNT  -> piece
--
-- ============================================================

START TRANSACTION;
```

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

```sql
-- Recipe ingredients -> canonical units (gram, milliliter, piece)
UPDATE recipe_ingredient
SET
  quantity = ROUND(
    CASE LOWER(TRIM(unit))
      WHEN 'gram' THEN quantity
      WHEN 'kilogram' THEN quantity * 1000

      WHEN 'milliliter' THEN quantity
      WHEN 'liter' THEN quantity * 1000
      WHEN 'teaspoon' THEN quantity * 5
      WHEN 'tablespoon' THEN quantity * 15
      WHEN 'cup' THEN quantity * 240

      ELSE quantity
    END, 2
  ),
  unit = CASE
    WHEN LOWER(TRIM(unit)) IN (
      'gram',
      'kilogram'
    )
      THEN 'gram'
    WHEN LOWER(TRIM(unit)) IN (
      'milliliter',
      'liter',
      'teaspoon',
      'tablespoon',
      'cup'
    )
      THEN 'milliliter'
    WHEN LOWER(TRIM(unit)) IN (
      'piece', 'whole',
      'clove', 'slice',
      'pinch'
    )
      THEN 'piece'
    ELSE LOWER(TRIM(unit))
  END;

-- Pantry -> canonical units (gram, milliliter, piece)
UPDATE pantry
SET
  quantity = ROUND(
    CASE LOWER(TRIM(unit))
```

```
      WHEN 'gram' THEN quantity
      WHEN 'kilogram' THEN quantity * 1000

      WHEN 'milliliter' THEN quantity
      WHEN 'liter' THEN quantity * 1000
      WHEN 'teaspoon' THEN quantity * 5
      WHEN 'tablespoon' THEN quantity * 15
      WHEN 'cup' THEN quantity * 240

      ELSE quantity
    END, 2
  ),
  unit = CASE
    WHEN LOWER(TRIM(unit)) IN (
      'gram',
      'kilogram'
    )
      THEN 'gram'
    WHEN LOWER(TRIM(unit)) IN (
      'milliliter',
      'liter',
      'teaspoon',
      'tablespoon',
      'cup'
    )
      THEN 'milliliter'
    WHEN LOWER(TRIM(unit)) IN (
      'piece', 'whole',
      'clove', 'slice',
      'pinch'
    )
      THEN 'piece'
    ELSE LOWER(TRIM(unit))
  END;

COMMIT;

SELECT 'unit normalization migration completed' AS status;
```

B. **Sample Queries**

**JOIN Queries (Multi-Table)**

SQL

**Recipe Ingredients with Names - RecipeIngredientDAO.java:**

```
SELECT ri.recipe_id, ri.ingredient_id, ri.quantity, ri.unit, i.name AS ingredient_name
FROM recipe_ingredient ri
JOIN ingredient i ON ri.ingredient_id = i.ingredient_id
WHERE ri.recipe_id = ?
ORDER BY i.name ASC
```

Purpose: Returns all ingredients for a recipe with ingredient names, avoiding a second query per ingredient.

SQL

**Pantry with Ingredient Names - PantryDAO.java:**

```
SELECT p.ingredient_id, p.user_id, p.quantity, p.unit, i.name AS ingredient_name
FROM pantry p
JOIN ingredient i ON p.ingredient_id = i.ingredient_id
WHERE p.user_id = ?
ORDER BY i.name ASC
```

Purpose: Shows user's pantry with human-readable ingredient names.

SQL

**Meal Entries with Recipe Details - MealEntryDAO.java (base fragment):**

```
SELECT me.recipe_id, me.user_id, me.meal_type, me.scheduled_date,
    r.title AS recipe_title, r.category AS recipe_category
FROM meal_entry me
JOIN recipe r ON me.recipe_id = r.recipe_id
WHERE me.user_id = ? AND me.scheduled_date BETWEEN ? AND ?
```

Purpose: Populates weekly planner without requiring separate recipe lookups.

## Range Queries (BETWEEN)

SQL

**Weekly Planner Range - MealEntryDAO.java:**

```
WHERE me.scheduled_date BETWEEN ? AND ?
ORDER BY me.scheduled_date, me.meal_type
```

Purpose: Efficiently loads entire week (or month) of meals in one query.

## Advanced Query Patterns

SQL

**INSERT ON DUPLICATE KEY UPDATE (Upsert) - PantryDAO.java:**

```
INSERT INTO pantry (ingredient_id, user_id, quantity, unit)
VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
  quantity = VALUES(quantity),
  unit = VALUES(unit)
```

Purpose: Atomically adds a pantry item or updates quantity if it already exists (no race condition between check and insert).

SQL

**Existence Checks (COUNT) - PantryDAO.java:**

```
SELECT COUNT(*) FROM pantry
WHERE ingredient_id = ? AND user_id = ?
```

Purpose: Efficient boolean check before insert to avoid constraint violations.

### Transactions

SQL

**Meal Entry Swap - MealEntryDAO.java**

```
connection.setAutoCommit(false);
try {
  deleteEntry(first);     // DELETE 1
  deleteEntry(second);    // DELETE 2
  addEntry(swapped1);     // INSERT 1
  addEntry(swapped2);     // INSERT 2
  connection.commit();
} catch (SQLException e) {
  connection.rollback();
  throw e;
} finally {
  connection.setAutoCommit(previousAutoCommit);
}
```

Purpose: Ensures all-or-nothing atomicity when swapping two occupied meal slots. If any step fails, all changes roll back.

C. **Frontend (GUI) Development**:

The frontend of Le Festin was developed using Java Swing, but the interface was designed to

feel more modern and organized than a typical desktop application. Instead of opening multiple windows, the system uses a CardLayout to switch smoothly between pages like the Recipe List and Pantry while keeping the sidebar and header fixed. This creates a cleaner and more seamless user experience.

## IV. Conclusion and Reflection

Building Le Festin from scratch was not an easy feat, from brainstorming to implementation. We had to think through so many layers — the database schema, the data access objects (DAOs), the business logic, and the UI. These pieces had to fit perfectly, and the setup took time.

The biggest challenge we faced was getting on track; a group member was sick, we had insufficient people and time to work. We had to take almost a week's worth of backlogs individually to finish within the remaining weeks. Next, managing the database connections between the UI and the DAO. Debugging database issues is slow because we have to trace from the UI all the way down to the SQL. The recipe matching algorithm was also tricky. Figuring out how to calculate which recipes matched the ingredients someone had in their pantry involved a lot of thinking. It’s not hard math, but getting it to feel right and work fast took patience. Lastly, the UI was surprisingly complex with multiple panels talking to each other, and state management became messy sometimes. We learned that small organizational choices at the start matter a lot.

We learned that structure matters more than we thought. Separating concerns into layers—models, DAOs, services, and UI made things easier. Testing small pieces as we built them would have saved us hours.

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

## V. Appendixes

### A. User manual for each feature

| FEATURE | STEPS |
|---|---|
| **Setup Wizard** | 1. Enter your MySQL Username<br>2. Enter your MySQL Password<br>3. Click Test Connection to verify credentials<br>4. Click Next<br>5. The schema will always be created automatically<br>6. Check Load built-in seed data if you want sample recipes and demo accounts (pre-checked by default)<br>a. Demo accounts: angela, carl, elizah — password: password123<br>7. Click Initialize & Finish<br>8. The wizard closes automatically on success, and the app loads |
| **Authentication (Login / Register)** | 1. Launch the app — the Login dialog appears.<br>2. To sign in: enter Username and Password → click Sign In.<br>3. To create an account: click Register → fill in Choose a username (min. 3 characters), Password (min. 6 characters), and Confirm password → click Create Account. The app logs you in automatically on success.<br>4. The main window opens, your username appears as a 👤 username button in the top-right. Click it to reveal the Logout option. |
| **Recipes (Browse / View / Add / Edit / Delete)** | 1. Open Recipes from the sidebar (default view on login).<br>2. Search by title or category using the search field.<br>3. Add a recipe:<br>○ Click Add Recipe (next to the search bar).<br>○ Fill in Recipe title, Total time, Category, and Steps to follow.<br>○ Add ingredients: click + Add ingredient, then set the quantity, unit (dropdown), and ingredient name per row.<br>○ Click Save Recipe.<br>4. Edit: click the ✎ icon on a recipe card → update the fields → click Update Recipe.<br>5. Delete: click the ✖ icon on a card → confirm deletion.<br>6. View details: click the recipe card body (not the icons) to see its ingredients and steps; click Back to return. |
| **Pantry (My Pantry)** | 1. Open My Pantry from the sidebar. |

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

| | |
|---|---|
| | 2. Add ingredient: click + Add (next to the search bar) → the Add Ingredient dialog opens → enter name, quantity, and unit (unit is auto-suggested based on the name) → click Add to Pantry.<br>3. Edit: click the ✎ icon on a card (or double-click the card) → update quantity and/or unit (ingredient name is locked) → click Update.<br>4. Remove: click the ✖ icon on a card → confirm removal.<br>5. To find recipes you can cook with your current pantry: click Match Recipes (next to the search bar) to jump to Suggestions. |
| **Recipe Suggestions (Recipe Matching)** | 1. Open Suggestions from the sidebar.<br>2. Suggestions load automatically when the panel opens — no manual refresh needed.<br>3. Use the filter buttons to narrow results: All / Ready to Cook / Partial Match.<br>4. Review each card: the progress bar shows the pantry match percentage; missing ingredients are listed under Still needed.<br>5. Assign a recipe to the meal planner: click Assign to Plan on a card → pick a date and meal slot (Breakfast / Lunch / Dinner) → confirm. |
| **Weekly Meal Planner** | 1. Open Meal Planner from the sidebar.<br>2. Navigate weeks with ◂ Prev week / Next week ▸.<br>3. Assign a recipe to a slot:<br>○ Click an empty slot → click Assign Recipe → choose a recipe → click Select Recipe.<br>○ Click a filled slot to get options: Change Recipe or Clear Slot.<br>○ Or assign directly from a Suggestions card via "Assign to Plan."<br>4. Drag a filled slot to another slot to move it (empty target) or swap it (filled target).<br>5. Auto-Generate Week: click Auto-Generate Week → confirm → empty slots are filled automatically from your recipes (existing assignments are kept).<br>6. Clear Week: click Clear Week → confirm to remove all entries for the current week.<br>7. Export CSV: click Export CSV (enabled only when the week has entries) → a file chooser opens to save the plan. |
| **Grocery List** | 1. Open Grocery List from the sidebar.<br>2. Adjust the From and To date spinners if needed (defaults to the current Monday–Sunday).<br>3. The list generates automatically on open. Click Generate List to refresh after changing the date range. |

<table>
<tr><td></td><td>
4. Results appear in a table: Ingredient | Quantity | Unit | Note — showing only the ingredients your pantry doesn't already cover for the planned meals.<br>
5. Export CSV: click Export CSV (enabled once results exist) → choose a file name and location (.csv extension is added automatically if omitted).<br>
6. Print: click Print (enabled once results exist) → the table is sent to a printer.
</td></tr>
<tr><td>CSV Export (Meal Plan & Grocery)</td><td>
1. From Grocery List: after generating a non-empty list, click Export CSV.<br>
2. From Weekly Planner: click Export CSV in the planner nav bar (enabled only when the week has entries).<br>
3. In the file chooser, pick a location and file name; .csv is appended automatically if omitted.<br>
4. On success:<br>
○ Grocery List shows a confirmation with the row count and the full saved file path.<br>
○ Weekly Planner shows a confirmation with the row count and filename only.<br><br>
Format<br>
Meal plan:<br>
None<br>
Date,Meal Type,Recipe Title,Prep Time (mins)<br>
2026-05-18,Breakfast,Classic Scrambled Eggs,10<br>
2026-05-18,Lunch,Garlic Butter Chicken,25<br>
2026-05-18,Dinner,Pork Adobo,60<br>
2026-05-19,Breakfast,Garlic Fried Rice,15<br><br>
Grocery list:<br>
None<br>
Ingredient,Quantity,Unit<br>
Bread,8,slice<br>
Butter,19,tablespoon<br>
Cheese,2,cup<br>
Chicken breast,12,piece
</td></tr>
</table>

**UNIVERSITY OF THE PHILIPPINES**
**Tacloban College**
**Division of Natural Sciences and Mathematics**
**Bachelor of Science in Computer Science**

## B. Application setup/installation instructions

On first launch, Le Festin opens a **Setup Wizard** that guides you through entering your MySQL credentials, creating the schema, and loading seed data — no manual configuration needed.

### Option 1: Local Development (Build from Source)

Use this option to run the app directly from the code.

#### 1. Clone the repository

```
None
git clone https://github.com/ausdotsn50/le-festin.git
cd le-festin/le_festin
```

#### 2. Compile

*Windows*

```
Java
javac -cp "lib/*" -d out src\*.java src\config\*.java src\dao\*.java src\dao\impl\*.java src\helper\Helper.java src\model\*.java src\service\*.java src\ui\*.java src\ui\dialogs\*.java src\ui\panels\*.java
```

*MacOS*

```
Java
javac -cp "lib/*" -d out src/*.java src/config/*.java src/dao/*.java src/dao/impl/*.java src/helper/Helper.java src/model/*.java src/service/*.java src/ui/*.java src/ui/dialogs/*.java src/ui/panels/*.java
```

#### 3. Run

*Windows*

```Java
java -cp "out;lib/*" Main
```

*MacOS*

```Java
java -cp "out:lib/*" Main
```

**Option 2: Running the Executables (.jar, . exe)**

1. Go to this GitHub repository and download it as a ZIP file.
2. Extract the files from the ZIP file.
3. Go to the installers directory.
4. Launch the application by double-clicking **le-festin.exe**. Alternatively, extract **jar_installer.zip,** then double-click **le-festin.jar** to launch the application.